

Neuromorphic Motion Segmentation with Spiking Neural Networks for Robotic Perception

Daniela Esparza * Sergio Trejo ** Gerardo Flores ***

* *Laboratorio de Percepción y Robótica [LAPyR], Center for Research in Optics, Loma del Bosque 115, 37150, Mexico (e-mail: daniesparza@cio.mx).*

** (e-mail: sergiotrejo@cio.mx)

*** *RAPTOR Lab, School of Engineering at the Texas A&M Int. Univ, 5201 University Boulevard, Laredo, TX 78041, US (e-mail: gerardo.flores@tamui.edu)*

Abstract: Event-based cameras have emerged as a promising technology for robotic applications, offering high temporal resolution and low latency. Motion segmentation is a critical task for dynamic environments, enabling essential capabilities such as obstacle avoidance, SLAM, and autonomous navigation. In this work, we propose a spiking neural network (SNN) architecture based on a U-Net encoder-decoder to perform motion segmentation using event streams captured by event-based cameras. The network processes spatio-temporal data efficiently and is trained with a combination of Dice loss and binary cross-entropy to handle imbalanced segmentation masks. We leverage the EVIMO dataset, which provides pixel-wise motion labels, egomotion data, and event streams, to train and evaluate the model. Our method achieves an Intersection over Union (IoU) score of 51.6%, comparable to state-of-the-art approaches like SpikeMS, while maintaining computational efficiency. Qualitative results show accurate pixel-wise segmentation of dynamic objects, demonstrating clear visual alignment between the predictions and the ground truth. These results highlight the potential of our approach for deployment in neuromorphic robotic systems requiring real-time processing.

Copyright © 2025 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: Motion segmentation, spiking neural networks, event-based cameras.

1. INTRODUCTION

Neuromorphic engineering provides an efficient solution for the speed-accuracy trade-off in tasks like segmentation and optical flow by reducing redundant pixel-level processing. This issue stems from traditional frame-based cameras, which capture images indiscriminately. In contrast, event-based cameras transmit information only during changes in light intensity, where each pixel independently generates asynchronous *events*. Dynamic objects and camera motion both trigger events, leading to a mix of background and object-related data. When the event stream contains few or no events, energy-efficient processing becomes crucial. Spiking Neural Networks (SNNs) provide a suitable solution by introducing the ability to encode temporal information. Unlike traditional Artificial Neural Networks (ANNs), SNNs utilize spiking neuron models that account for the temporal intervals between spikes, enabling energy-efficient computations within a highly parallelizable framework.

This paper explores the integration of event-based cameras with Spiking Neural Networks to address motion segmentation demonstrating their potential for efficient and accurate processing in neuromorphic vision systems.

1.1 Motivation

The principal motivation of this work is to contribute to obstacle avoidance, object tracking, and SLAM systems for Unmanned Aerial Vehicles (UAVs). Talking about SLAM, authors such as Esparza and Flores (2022) focus on dynamic objects using RGB cameras; however, event-based SLAMs consider static object environments, as in Vidal et al. (2018). In these applications, motion segmentation serves as a critical first stage, enabling the distinction between static and dynamic objects. Motion segmentation separates dynamic objects from the background, but traditional synchronous systems face speed limitations for real-time applications. To address this limitation, recent advances have focused on asynchronous computing systems, which offer improved data processing by capturing events as they occur.

1.2 Problem Statement and Contribution

Dynamic environments in tasks such as SLAM and autonomous driving require an efficient distinction between static and moving objects. This raises a critical question: can event-based cameras provide sufficient and reliable information to detect and segment moving objects in such scenarios?

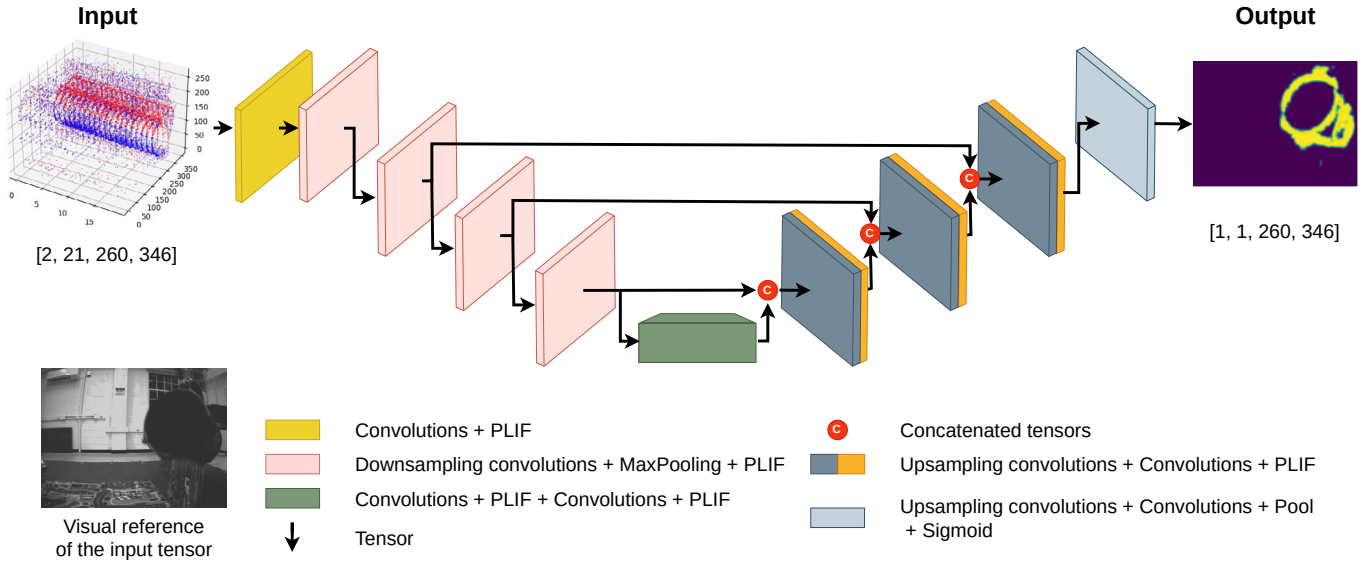


Fig. 1. The input to the SNN is a tensor representing a 21 ms time window, consisting of two event channels (positive and negative) and the sensor dimensions (346×260). The process starts with a convolutional layer combined with PLIF neurons, followed by an encoder that progressively reduces the temporal and spatial dimensions of the tensor. The compressed representation passes through a bottleneck layer for further processing. During the decoding phase, upsampling layers merge outputs from previous decoder layers with corresponding encoder outputs via concatenation. The final output tensor is generated using additional convolutional layers and a sigmoid activation function. A binary pixel-wise object mask serves as the ground truth, identifying the moving objects for each input tensor.

To address this question, we propose a deep Spiking Neural Network (SNN) with an encoder-decoder architecture specifically designed for motion segmentation in event streams captured by a monocular event-based camera. The primary goal is to classify events into two categories: (1) those originating from independently moving dynamic objects, and (2) those caused by the motion of the camera relative to static elements in the background.

The main contributions of this work are as follows:

- A novel SNN architecture optimized for fast and efficient motion segmentation, enabling real-time applications such as obstacle avoidance.
- A 3D encoding of input events that captures temporal information, significantly improving segmentation accuracy.
- A hardware-efficient downsampling technique leveraging maximum pooling, which enhances model performance while maintaining computational efficiency.
- A supervised training framework using the EV-IMO dataset, which includes pixel-wise object masks and event streams across five diverse scenes.

2. RELATED WORK

Motion segmentation using event-based cameras has recently emerged as a promising approach for dynamic scene understanding, gaining significant attention in works such as Stoffregen et al. (2019), Falanga et al. (2020), Parameshwara et al. (2020), Parameshwara et al. (2021b), He et al. (2021), Zhou et al. (2021) and Zhou et al. (2023). Various studies in the literature have explored different methods to tackle this challenge, with many leveraging Artificial Neural Network (ANN) architectures, as demonstrated in

Mitrokhin et al. (2020), Zhang et al. (2023), and Rahmon et al. (2021). A prevalent approach involves preprocessing event streams into image-like frame representations optimized for ANN inputs, as seen in works like Maqueda et al. (2018) and Sanket et al. (2020).

Zhang et al. (2023) introduced a multi-scale recurrent neural network (MSRNN) for motion segmentation. This method processes the event stream by converting it into a voxel grid representation, preserving the temporal information required for segmentation tasks. Building on a convolutional neural network (CNN) architecture, MSRNN effectively leverages both spatial and temporal features of event data. Similarly, Mitrokhin et al. (2020) proposed representing the event stream as a 3D point cloud, where the spatial coordinates x and y and the temporal dimension are treated as features. The point cloud is then processed through a Graph Convolutional Network, enabling robust segmentation of motion in dynamic scenes.

Another noteworthy contribution is Wang et al. (2025), which proposed an unsupervised learning framework called Un-EVIMO. This method uses a CNN-based architecture to predict binarized masks of independently moving objects. The input to the network consists of temporal bins, which segment the event stream into discrete intervals for efficient processing. Similarly, Georgoulis et al. (2024) employs temporal bins with spatial dimensions as inputs. Their approach comprises three stages: ego-motion compensation, optical flow estimation, and motion segmentation, all of which are implemented using CNN-based architectures.

Inspired by these works, we also use temporal bins as the input representation for our network. However, unlike

the methods described above, our approach introduces a spiking neural network (SNN) architecture specifically designed for efficient motion segmentation. This architecture not only preserves temporal and spatial information but also opens the possibility of implementation on neuromorphic hardware, offering significant advantages in terms of computational efficiency and hardware compatibility for future applications.

3. METHODS

3.1 Event Data Representation and Preprocessing

Event-based cameras produce asynchronous events, denoted as e_i , whenever the change in luminance at a specific pixel exceeds a predefined threshold. Each event is described by:

$$e_i = (x_i, y_i, t_i, p_i), \quad (1)$$

where (x_i, y_i) denote the pixel coordinates, t_i is the timestamp of the event, and p_i represents the polarity, which indicates whether the change in luminance is positive (+1) or negative (−1).

Despite the asynchronous nature of event-based cameras, modern computers operate on clock-driven, synchronous systems. To accommodate this discrepancy during training, we preprocess the event stream into a discrete-time representation. Specifically, we segment the continuous event stream into fixed-duration time windows, forming what we call *input stream*. These streams are represented as a tensor of size (C, B, H, W) , where:

- $C = 2$ represents the two input channels (positive and negative events),
- B is the number of temporal slices within the time window,
- H and W correspond to the resolution of the event camera, representing the number and spatial arrangement of pixels.

Event streams encode the number of positive and negative events triggered at each pixel during the time window. This preprocessing step transforms the asynchronous event data into a format compatible with synchronous computing systems, enabling the use of modern machine learning frameworks. By preserving both the spatial and temporal information of the events, this representation ensures that the dynamic characteristics of the scene are effectively encoded for further processing.

3.2 Neural Network Architecture

Our work builds upon the Spiking Neural Network (SNN) proposed by Cuadrado et al. (2023), which is inspired by the U-Net architecture. This network consists of three main phases: an encoder, a bottleneck, and a decoder. The input tensor has a shape of

$$[\text{Batch}, \text{Polarity}, \text{Bins}, \text{Height}, \text{Width}], \quad (2)$$

where the polarity represents the two channels for positive and negative events, and the time dimension captures temporal information within a specific window. After processing, the output tensor reduces to

$$[\text{Batch}, 1, \text{Height}, \text{Width}], \quad (3)$$

holding only the moving events in the spatial dimensions. The network was trained using supervised learning on the EV-IMO dataset Mitrokhin et al. (2019), which provides pixel-wise ground truth for motion segmentation.

The event-based camera captures asynchronous events, which are preprocessed into the appropriate input tensor format. The network begins with an initial 3D convolution stage, increasing the number of channels to 32 while preserving the original tensor size. During the encoder phase, each stage applies a sequence of operations: a 3D convolution that maintains the tensor's size, followed by an increase in the number of channels (doubling at each stage), spatial downsampling using 3D max pooling to halve the tensor's height and width, and a Parametric Leaky Integrate-and-Fire (PLIF) activation function as proposed by Fang et al. (2021b).

The bottleneck, according to Fang et al. (2021a), represents the most compressed form of the encoded tensor, enabling efficient processing and feature extraction. During the decoder phase, the operations are reversed. Each stage begins by doubling the spatial dimensions of the tensor using nearest-neighbor upsampling. This is followed by a pure 3D transposed convolution that maintains the tensor's shape, then a second 3D transposed convolution that halves the number of channels, and finally, a spiking neuron layer that processes the output.

This architecture effectively captures both spatial and temporal features of event streams, enabling robust motion segmentation. By leveraging the PLIF activation and 3D convolutional operations, the network achieves computational efficiency and accuracy, laying the foundation for future deployment on neuromorphic hardware.

3.3 Dataset

For this work, we utilize the EV-IMO dataset proposed by Mitrokhin et al. (2019), which is specifically designed for motion segmentation tasks using event-based cameras. This dataset includes both indoor and outdoor scenarios captured with a DAVIS 346 event-based camera, providing pixel-wise object masks as labels. In addition to event data, the EV-IMO dataset offers accurate pixel-wise motion masks, egomotion information, and ground truth depth maps, making it highly suitable for supervised learning applications.

The input data for the network are preprocessed by aggregating events triggered within 20 ms in 21 contiguous matrices inside a tensor. This preprocessing step transforms the asynchronous event data into a structured representation compatible with the network. For the ground truth, the motion masks provided by the dataset are binarized, where pixels labeled as '1' represent dynamic objects, and pixels labeled as '0' correspond to background events. Each event is represented as (1).

For GPU-based simulation, events are accommodated into a 5-dimensional tensor with the shape given by (2). The last three dimensions represent the event adapted over temporal slices. Specifically:

- The **bin dimension** records the single occurrence of events over the temporal window.

- The **spatial dimensions** (*Height, Width*) capture the spatial distribution of events in the 2D grid.

It is important to note that the resulting tensor from event accumulation is not equivalent to an actual image or a sequence of images but rather a structured representation of spatiotemporal event activity.

This preprocessing approach preserves the asynchronous and sparse nature of the event data while structuring it into a format compatible with modern machine learning frameworks, enabling effective training of our neural network for motion segmentation.

3.4 Loss Function

For supervised training, we employ a combined loss function comprising Dice loss (4) and binary cross-entropy (BCE) (5), both well-suited for segmentation tasks in neural networks Jadon (2020); Taghanaki et al. (2018). Dice loss evaluates the similarity between predicted and ground truth segmentation masks, while BCE focuses on accurate class separation. The combination leverages Dice loss's robustness against class imbalance and BCE's precision in binary classification.

The Dice loss is defined as:

$$loss_{dice} = 1 - \left(\frac{2 \sum_i^N p_i g_i + \epsilon}{\sum_i^N p_i + \sum_i^N g_i + \epsilon} \right) \quad (4)$$

where p_i denotes the predicted probability, g_i represents the ground truth, and ϵ is a small constant to prevent division by zero. A higher similarity between p_i and g_i increases the numerator, thereby reducing the loss. This makes Dice loss particularly effective in handling imbalanced segmentation masks.

The binary cross-entropy loss is expressed as:

$$loss_{bce} = -g_i \log p_i + (1 - g_i) \log (1 - p_i) \quad (5)$$

Here, p_i is the predicted probability of the first class (e.g., dynamic events), and $1 - p_i$ represents the background class. Similarly, g_i and $1 - g_i$ correspond to the ground truth for the respective classes. BCE evaluates the prediction for both classes, ensuring balanced contributions to the loss function.

The total loss function combines these components as:

$$loss_{tot} = \lambda loss_{bce} + loss_{dice} \quad (6)$$

where λ is a weighting factor used to balance the contributions of BCE and Dice loss. This approach ensures that the loss function captures both pixel-level accuracy and class-level balance, making it robust and effective for segmentation tasks.

3.5 Training Procedure

The network processes a spatio-temporal 3D matrix in a single step using a non-shuffled dataset and regular back-propagation, implemented with the SpikingJelly Python library Fang et al. (2023). Temporal context is managed through 3D convolutions "whole mode," ensuring that the model produces an output tensor of the same size as the input, regardless of the temporal window length.

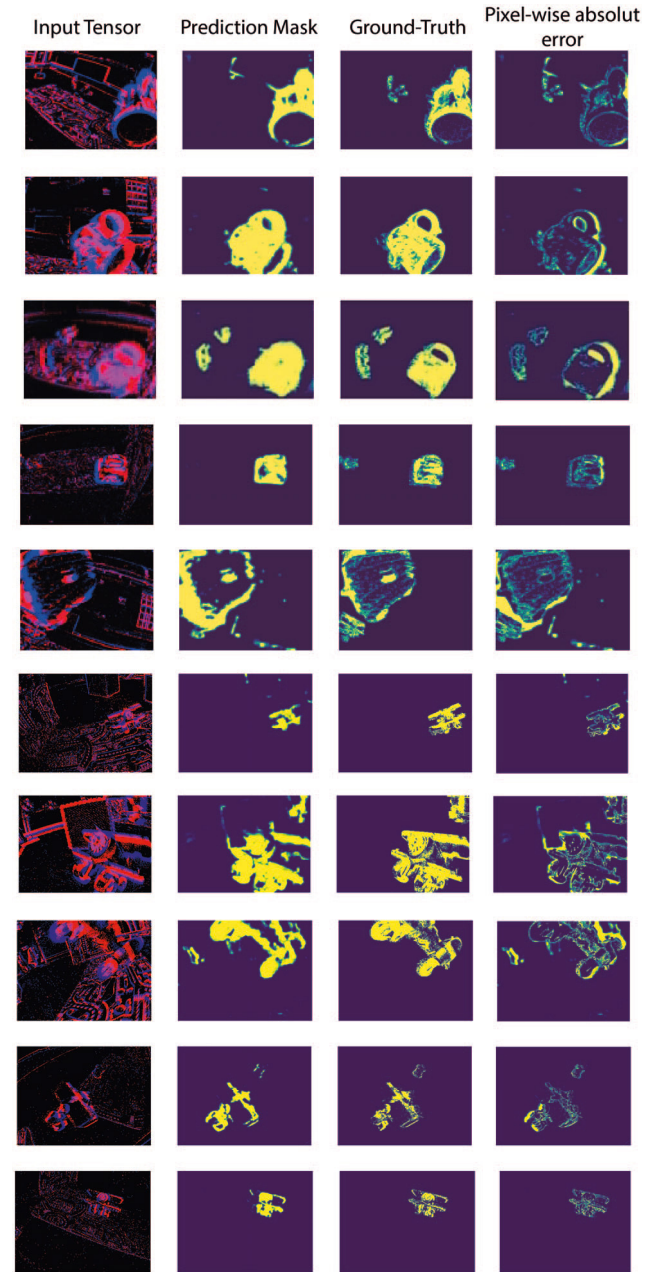


Fig. 2. From left to right. The first column shows a representation of the input tensor. The next column corresponds to the prediction from SNN (yellow pixels show the dynamic points). The ground truth is shown in the third column. Finally, the last column corresponds to the difference between prediction and ground truth.

Training utilizes an exponential learning rate scheduler with steps at 10, 20, 40, and 80 epochs, and a decay factor of $\gamma = 0.5$. The optimizer used was AdamW proposed by Loshchilov and Hutter (2017) with a learning rate of $1e-3$ and a weight decay of $1e-3$. For training, we use a batch size of 1. The time window chosen for the training process was 20 ms.

4. RESULTS

We evaluated the proposed SNN using the EV-IMO dataset. For the discretized events, we carried out three

Table 1. Comparison of IoU

Dataset EV-IMO	Wall			Floor			Fast			Box			Table		
Time window	100	50	20	100	50	20	100	50	20	100	50	20	100	50	20
ANN/CNN															
EV-IMO	-	-	78±5	-	-	59±9	-	-	67±3	-	-	70±5	-	-	79±6
GConv	83±4	-	80±7	79±7	-	55±19	74±17	-	39±19	81±8	-	60±18	80±7	-	51±16
EVDodgeNet	-	-	72±9	-	-	61±6	-	-	60±10	-	-	67±8	-	-	70±8
Un-EVIMO	-	-	53±19	-	-	56±15	-	-	44±21	-	-	45±24	-	-	50±21
SNN															
SpikeMS	65±7	-	63±6	60±5	-	53±16	45±11	-	38±10	61±7	-	65±8	52±13	-	50±8
Ours	26±4	55±2	59±2	30±7	64±2	49±4	30±13	48±6	49±6	26±6	46±2	47±2	28±10	50±4	49±3

experiments changing the time window: 20 ms, 50 ms, and 100 ms. Sequences contain objects moving in different directions by changing velocities of movement. As a result, objects in the scene may appear larger or smaller depending on the camera and object movements.

4.1 Qualitative evaluation

The following figures are broken down into columns. We represent the input to the SNN as a compact image for ease of understanding. The blue dots correspond to negative events and the red dots to positive events.

The second column represents the SNN prediction. To be better appreciated, the yellow dots represent moving objects, and the violet pixels correspond to non-moving objects. The ground truth object is shown in the next column. Finally, we calculate the difference between the prediction and the ground truth. Note that the thick edges produced by certain movements cause a considerable error. Figure 2 shows the qualitative results.

4.2 Quantitative Results

We compare our test results with the ANN, CNN, and SNN approaches. EVIMO, GConv, and EvDodgeNet are based on ANN architectures. Un-EVIMO works with CNNs. For SNNs, SpikeMS and our proposal are evaluated.

For the comparison, we took the values of the SpikeMS, Un-EVIMO, EVDodgeNet, GConv, and EVIMO from Parameshwara et al. (2021a), Wang et al. (2025), and Mitrokhin et al. (2020). The time window for evaluating EVIMO, Un-EVIMO, and EVDodgeNet was 25 ms, while the results of GConv and SpikeMS were 20 ms and 100 ms. Although our proposed architecture must have 21 bins, we can fit different time windows to the bins. For the experiments, we tested our proposal with time windows of 20 ms, 50 ms, and 100 ms. Table 1 shows the results.

While the IoU values of our work do not achieve 100% accuracy in the testing process, they are comparable to state-of-the-art SNN and CNN results. Furthermore, our method shows a lower variance in IoU than the existing approaches, indicating greater consistency. Our results are similar when testing time windows are closer to the training process. We attribute these results to the spike saturation in the bins. On the other hand, table 1 shows that ANNs perform better than SNN architectures. This could be because the training process is more adequate.

A limitation of the proposed network is its dependence on the size of the camera sensor. Although newer camera models are entering the market, the availability of motion segmentation datasets remains limited.

5. CONCLUSION

We proposed a spiking neural network (SNN) for motion segmentation, designed with the potential to extend its application to tasks such as SLAM, obstacle avoidance, and other time-critical scenarios. To accommodate the fast data processing requirements of UAVs, where rapid motion demands high temporal resolution, we trained and tested the network using short time windows, enabling the processing of multiple windows per second.

Our experiments, conducted both qualitatively and quantitatively, demonstrate promising results. Preliminary evaluations show that our approach achieves pixel-wise segmentation with a visual difference between predictions and ground truth. Quantitative comparisons with SpikeMS indicate that our method achieves an Intersection over Union (IoU) metric close to the SNN state-of-the-art. However, the ANN and CNN results show a long way to go for the SNN approaches.

Frame-based cameras struggle in low-light conditions. In contrast, event-based cameras offer a significant advantage due to their high dynamic range, allowing them to detect events even in low-illumination environments. This capability makes them well-suited for motion segmentation, a critical task for UAVs operating in varied lighting and challenging scenarios.

The same experiments with the EV-IMO dataset were conducted on embedded platforms to compare their processing time against PC-based implementations. In our setup, the PC experiments inferred segmentation masks in 0.09s, whereas the Jetson Xavier AGX executed the neural network with a prediction time of 0.65s per inference, making the PC approximately 8 times faster than the Jetson. However, further experiments on newer NVIDIA-embedded platforms could enhance computational efficiency and reduce processing time, potentially bridging the performance gap between embedded and PC-based implementations.

Looking ahead, several improvements could enhance the applicability of the proposed SNN. Additionally, while we trained and tested the network using the EVIMO dataset, incorporating additional datasets such as Sanket et al. (2020) could further enhance its robustness. Lastly, the current implementation supports tensors from a specific camera resolution (240 × 346). Adapting the network to accommodate different input sizes would enable compatibility with various event-based camera models, broadening its applicability across platforms and devices.

REFERENCES

- Cuadrado, J., Rançon, U., Cottureau, B.R., Barranco, F., and Masquelier, T. (2023). Optical flow estimation from event-based cameras and spiking neural networks. *Frontiers in Neuroscience*, 17.
- Esparza, D. and Flores, G. (2022). The STDynSLAM: A stereo vision and semantic segmentation approach for VSLAM in dynamic outdoor environments. *IEEE Access*, 10, 18201–18209. doi: 10.1109/ACCESS.2022.3149885.
- Falanga, D., Kleber, K., and Scaramuzza, D. (2020). Dynamic obstacle avoidance for quadrotors with event cameras. *Science Robotics*, 5(40), eaaz9712.
- Fang, W., Chen, Y., Ding, J., Yu, Z., Masquelier, T., Chen, D., Huang, L., Zhou, H., Li, G., and Tian, Y. (2023). Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40), eadi1480. doi:10.1126/sciadv.adi1480.
- Fang, W., Yu, Z., Chen, Y., Huang, T., Masquelier, T., and Tian, Y. (2021a). Deep residual learning in spiking neural networks. *Advances in Neural Information Processing Systems*, 34, 21056–21069.
- Fang, W., Yu, Z., Chen, Y., Masquelier, T., Huang, T., and Tian, Y. (2021b). Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2661–2671.
- Georgoulis, S., Ren, W., Bochicchio, A., Eckert, D., Li, Y., and Gawel, A. (2024). Out of the room: Generalizing event-based dynamic motion segmentation for complex scenes. *2024 International Conference on 3D Vision (3DV)*, 442–452.
- He, B., Li, H., Wu, S., Wang, D., Zhang, Z., Dong, Q., Xu, C., and Gao, F. (2021). Fast-dynamic-vision: Detection and tracking dynamic objects with event and depth sensing. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3071–3078. IEEE.
- Jadon, S. (2020). A survey of loss functions for semantic segmentation. *2020 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*, 1–7.
- Loshchilov, I. and Hutter, F. (2017). Decoupled weight decay regularization. In *International Conference on Learning Representations*.
- Maqueda, A.I., Loquercio, A., Gallego, G., García, N., and Scaramuzza, D. (2018). Event-based vision meets deep learning on steering prediction for self-driving cars. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 5419–5427.
- Mitrokhin, A., Hua, Z., Fermüller, C., and Aloimonos, Y. (2020). Learning visual motion segmentation using event surfaces. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14402–14411. doi:10.1109/CVPR42600.2020.01442.
- Mitrokhin, A., Ye, C., Fermüller, C., Aloimonos, Y., and Delbruck, T. (2019). EV-IMO: Motion segmentation dataset and learning pipeline for event cameras. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 6105–6112. doi: 10.1109/IROS40897.2019.8968520.
- Parameshwara, C.M., Li, S., Fermüller, C., Sanket, N.J., Evanusa, M.S., and Aloimonos, Y. (2021a). SpikeMS: Deep spiking neural network for motion segmentation. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3414–3420. doi: 10.1109/IROS51168.2021.9636506.
- Parameshwara, C.M., Sanket, N.J., Gupta, A., Fermüller, C., and Aloimonos, Y. (2020). MOMS with events: Multi-object motion segmentation with monocular event cameras. *arXiv preprint arXiv:2006.06158*, 2(3), 5.
- Parameshwara, C.M., Sanket, N.J., Singh, C.D., Fermüller, C., and Aloimonos, Y. (2021b). 0-MMS: Zero-shot multi-motion segmentation with a monocular event camera. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 9594–9600. IEEE.
- Rahmon, G., Bunyak, F., Seetharaman, G., and Palaniappan, K. (2021). Motion U-Net: Multi-cue encoder-decoder network for motion segmentation. In *2020 25th International conference on pattern recognition (ICPR)*, 8125–8132. IEEE.
- Sanket, N.J., Parameshwara, C.M., Singh, C.D., Kurutukulam, A.V., Fermüller, C., Scaramuzza, D., and Aloimonos, Y. (2020). EVDodgeNet: Deep dynamic obstacle dodging with event cameras. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 10651–10657. doi:10.1109/ICRA40945.2020.9196877.
- Stoffregen, T., Gallego, G., Drummond, T., Kleeman, L., and Scaramuzza, D. (2019). Event-based motion segmentation by motion compensation. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 7243–7252.
- Taghanaki, S.A., Zheng, Y., Zhou, S.K., Georgescu, B., Sharma, P.S., Xu, D., Comaniciu, D., and Hamarneh, G. (2018). Combo loss: Handling input and output imbalance in multi-organ segmentation. *Computerized medical imaging and graphics : the official journal of the Computerized Medical Imaging Society*, 75, 24–33.
- Vidal, A.R., Rebecq, H., Horstschaef, T., and Scaramuzza, D. (2018). Ultimate SLAM? combining events, images, and IMU for robust visual SLAM in HDR and high speed scenarios. In *IEEE Robotics and Automation Letters (RA-L)*.
- Wang, Z., Guo, J., and Daniilidis, K. (2025). Un-EVIMO: Unsupervised event-based independent motion segmentation. In A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, and G. Varol (eds.), *Computer Vision – ECCV 2024*, 228–245. Springer Nature Switzerland, Cham.
- Zhang, S., Sun, L., and Wang, K. (2023). A multi-scale recurrent framework for motion segmentation with event camera. *IEEE Access*, 11, 80105–80114. doi: 10.1109/ACCESS.2023.3299597.
- Zhou, Y., Gallego, G., Lu, X., Liu, S., and Shen, S. (2021). Event-based motion segmentation with spatio-temporal graph cuts. *IEEE transactions on neural networks and learning systems*, 34(8), 4868–4880.
- Zhou, Z., Wu, Z., Bouteau, R., Yang, F., Demonceaux, C., and Ginjac, D. (2023). RGB-Event fusion for moving object detection in autonomous driving. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 7808–7815.